

Corso di Laurea triennale in Ingegneria e Scienze Informatiche

Image Denoising Technique with Neural Network

Relatore:

Prof. Lazzaro Damiana

Co/Contro Relatore

Dott. Ezio Greggio

Candidato:

Matteo Vanni

Matricola: 0000935584

Contents

1	Articolanzio	9
1.1	Panoramica sul problema	9
1.2	Utilizzo di modelli di Deep Learning	9
1.3	Dataset utilizzati	9
1.3.1	Tensorflow: Oxford-IIIT Pet	9
1.3.2	Kvasir dataset	10
2	Rumore	11
3	Decsrizione delle reti	13
3.1	Autoencoder	13
3.2	RIDNet	15
4	Analisi ed ottimizzazione	19
4.1	Prestazioni	19
4.2	Analisi dei modelli	20
4.2.1	Autoencoder	20
4.2.2	RIDNet	21
4.3	Prima run	22
4.3.1	Autoencoder	22
4.3.2	RIDNet	24
4.4	Kvasir dataset	25
4.5	Terza run	25
4.6	Quarta run	25
4.7	Quantizzazione dei modelli	25
4.7.1	Performance su dataset sconosciuti	25

List of Figures

4.1	First training of the autoencoder	23
4.2	Autoencoder psnr level, pet dataset	23
4.3	First training of the autoencoder	24
4.4	First psnr of the autoencoder	24
4.5	PSNR value for kvasir dataset	25

Listings

2.1	Funzione per aggiungere rumore alle immagini	11
3.1	Keras Autoencoder Build	14
3.2	Keras Autoencoder Build	15

1 Articolanzio

1.1 Panoramica sul problema

L'immagine denoising è il processo di rimozione di rumore da un'immagine.

Il rumore, che è causato da svariate fonti, quali foto fatte in condizioni di scarsa illuminazione o problemi che corrompono i file, causa perdita d'informazione sull'immagine.

Cos'è il rumore? Un'aggiunta casuale di pixel che non appartengono all'immagine originale e ce ne sono di varie tipologie:

Impulse Noise(IN) dove i pixel sono completamente diversi da quelli attorno. Esistono due categorie di IN: Salt and Pepper Noise(SPN) e Random Valued Impulse Noise(RVIN).

Additive White Gaussian Noise(AWGN) cambia ogni pixel dall'originale di una piccola quantità.

1.2 Utilizzo di modelli di Deep Learning

È essenziale rimuovere il rumore e ristabilire l'immagine originale dove riottenere l'immagine originale è importante per prestazioni robuste o ricostruire le informazioni mancanti è molto utile, come immagini astronomiche di oggetti molto lontani.

Le reti neurali convoluzionali lavorano bene con le immagini e ne utilizzeremo N, menzionate in alcuni paper di ricerca e compareremo i risultati di ogni modello.

1.3 Dataset utilizzati

1.3.1 Tensorflow: Oxford-IIIT Pet

Il primo dataset utilizzato per l'addestramento dei modelli è stato Oxford-IIIT Pet, fornito da TensorFlow[1]. La scelta è stata motivata dalla natura delle immagini contenute, prevalentemente raffiguranti animali, le quali presentano un alto livello di dettaglio rispetto a immagini generiche. Questa caratteristica risulta particolarmente utile nel contesto della ricostruzione di immagini degradate, poiché l'aggiunta di rumore comporta una maggiore complessità nel processo di recupero dell'informazione visiva. Inoltre, il dataset consente l'utilizzo di risoluzioni superiori a 32×32 pixel, permettendo così l'addestramento e la valutazione dei modelli su immagini di dimensioni maggiori. dataset(colonscopie).

1.3.2 Kvasir dataset

Il secondo dataset impiegato per valutare l'affidabilità e la robustezza delle reti neurali è il dataset Kvasir[2], il quale comprende 1.000 immagini endoscopiche ad alta risoluzione, in particolare raffiguranti polipi e altre anomalie del tratto gastrointestinale. Sviluppato per supportare la ricerca nel campo della diagnosi assistita da computer (CADx), con l'obiettivo di migliorare l'accuratezza e la tempestività dell'identificazione di patologie gastrointestinali.

2 Rumore

Il rumore in un'immagine rappresenta un'alterazione dei valori dei pixel che si discosta dall'informazione visiva originaria. Esso può essere introdotto in diverse fasi del processo di acquisizione, trasmissione o compressione, ed è solitamente causato da limitazioni fisiche dei sensori, fluttuazioni ambientali o errori di quantizzazione. Il rumore compromette la qualità visiva e l'integrità semantica dell'immagine, rendendo più complessi compiti computazionali quali la classificazione, la segmentazione o la ricostruzione. Tra i modelli di rumore più comuni si annoverano il rumore gaussiano (additivo e a distribuzione normale), il rumore impulsivo (salt-and-pepper), e il rumore Poissoniano (associato alla statistica fotonica). La modellazione e la gestione del rumore sono elementi centrali nella computer vision, in particolare nei contesti di image denoising, compressione con perdita e generazione di immagini sintetiche in scenari controllati.

La prima funzione di rumore è u'aggiunta di rumore casuale a ogni pixel dell'immagine.

Listing 2.1: Funzione per aggiungere rumore alle immagini

```

1 def add_noise(x, noise_factor=0.1):
2     """
3     x: immagine in input
4     noise_factor: fattore di rumore da aggiungere
5     Ritorna:
6         x_noisy: immagine con rumore aggiunto
7     """
8     noise = tf.random.normal(shape=tf.shape(x), mean=0.0, stddev
9                             ↪ =1.0)
10    x_noisy = x + noise_factor * noise
11    return x_noisy

```

[illegible]

3 Decsrizione delle reti

3.1 Autoencoder

È stato implementato un autoencoder convoluzionale con architettura simmetrica encoder-decoder, progettato per apprendere una rappresentazione compatta delle immagini in input e successivamente ricostruirle con la massima fedeltà possibile. L'implementazione prevede in input immagini RGB.

Sono utilizzati strati convoluzionali a kernel 3×3 con attivazione ReLU e padding 'same', al fine di preservare la dimensionalità spaziale durante l'elaborazione.

La fase di codifica è composta da due blocchi sequenziali di convoluzione seguiti da max-pooling, che riducono progressivamente la risoluzione spaziale mantenendo e trasformando le caratteristiche salienti. La codifica appresa rappresenta una versione compressa dell'immagine originale.

La fase di decodifica riflette simmetricamente la struttura dell'encoder, utilizzando operazioni di upsampling seguite da convoluzioni per ricostruire la mappa spaziale originale. L'ultimo strato utilizza un'attivazione sigmoid su 3 canali, coerente con l'intervallo $[0,1]$ delle immagini RGB normalizzate.

Il modello è stato compilato con ottimizzatore Adam e funzione di perdita basata sulla binary cross-entropy, scelta comunemente in contesti di image reconstruction su immagini normalizzate. Il training mira a minimizzare la differenza pixel-wise tra l'immagine di input e quella ricostruita.

Listing 3.1: Keras Autoencoder Build

```

1 def build_autoencoder(input_shape):
2     """
3     Costruisce un autoencoder convoluzionale
4     Parametri:
5         input_shape: forma dell'immagine in input (es. (32,32,3))
6     Ritorna:
7         autoencoder: modello compilato
8     """
9     input_img = Input(shape=input_shape)
10
11     # Encoder
12     x = Conv2D(64, (3,3), activation='relu', padding='same')(
13         ↪ input_img)
14     x = MaxPooling2D((2,2), padding='same')(x)
15     x = Conv2D(64, (3,3), activation='relu', padding='same')(x)
16     encoded = MaxPooling2D((2,2), padding='same')(x)
17
18     # Decoder
19     x = Conv2D(64, (3,3), activation='relu', padding='same')(
20         ↪ encoded)
21     x = UpSampling2D((2,2))(x)
22     x = Conv2D(64, (3,3), activation='relu', padding='same')(x)
23     x = UpSampling2D((2,2))(x)
24     decoded = Conv2D(3, (3,3), activation='sigmoid', padding='
25         ↪ same')(x)
26
27     autoencoder = Model(input_img, decoded)
28     autoencoder.compile(optimizer='adam', loss='
29         ↪ binary_crossentropy', metrics=['accuracy'])
30
31     return autoencoder
32
33 #Creazione autoencoder
34 autoencoder = build_autoencoder(input_shape=(img_size[0],
35     ↪ img_size[1], 3))
36 autoencoder.summary()

```

3.2 RIDNet

stessa cosa dell'autocoder (vedere se aggiungere tutto il codice gigante del setup)

Listing 3.2: Keras Autoencoder Build

```

1  # MeanShift: sottrae (o aggiunge) la media RGB
2  @register_keras_serializable("MeanShift")
3  class MeanShift(Layer):
4      def __init__(self, rgb_mean, sign=-1, **kwargs):
5          """
6          rgb_mean: tupla con la media dei canali R, G, B.
7          sign: -1 per sottrarre la media, 1 per aggiungerla.
8          """
9          super(MeanShift, self).__init__(**kwargs)
10         self.rgb_mean = tf.constant(rgb_mean, dtype=tf.float32)
11         self.sign = sign
12
13         def call(self, x):
14             """
15             x      atteso in formato (batch, altezza, larghezza, 3)
16             """
17             return x + self.sign * self.rgb_mean
18
19 # BasicBlock: Conv2D seguita da attivazione ReLU
20 @register_keras_serializable("BasicBlock")
21 class BasicBlock(Layer):
22     def __init__(self, out_channels, kernel_size=3, stride=1,
23         ↪ use_bias=False, **kwargs):
24         super(BasicBlock, self).__init__(**kwargs)
25         self.conv = Conv2D(out_channels, kernel_size, strides=
26             ↪ stride,
27             padding='same', use_bias=use_bias)
28         self.relu = ReLU()
29
30         def call(self, x):
31             return self.relu(self.conv(x))
32
33 # ResidualBlock: due convoluzioni con skip connection
34 @register_keras_serializable("ResidualBlock")
35 class ResidualBlock(Layer):
36     def __init__(self, out_channels, **kwargs):
37         super(ResidualBlock, self).__init__(**kwargs)
38         self.conv1 = Conv2D(out_channels, 3, strides=1, padding='
39             ↪ same')
40         self.relu = ReLU()
41         self.conv2 = Conv2D(out_channels, 3, strides=1, padding='
42             ↪ same')
43
44         def call(self, x):
45             residual = self.conv1(x)

```

```

42     residual = self.relu(residual)
43     residual = self.conv2(residual)
44     out = Add()(x, residual)
45     return ReLU()(out)
46
47 # EResidualBlock: versione estesa con convoluzioni a gruppi
48 @register_keras_serializable("EResidualBlock")
49 class EResidualBlock(Layer):
50     def __init__(self, out_channels, groups=1, **kwargs):
51         super(EResidualBlock, self).__init__(**kwargs)
52         self.conv1 = Conv2D(out_channels, 3, strides=1, padding='
53             ↪ same', groups=groups)
54         self.relu = ReLU()
55         self.conv2 = Conv2D(out_channels, 3, strides=1, padding='
56             ↪ same', groups=groups)
57         self.conv3 = Conv2D(out_channels, 1, strides=1, padding='
58             ↪ valid')
59
60     def call(self, x):
61         residual = self.conv1(x)
62         residual = self.relu(residual)
63         residual = self.conv2(residual)
64         residual = self.relu(residual)
65         residual = self.conv3(residual)
66         out = Add()(x, residual)
67         return ReLU()(out)
68
69 # Merge_Run_dual: due rami convoluzionali con dilatazioni diverse
70 ↪ , poi merge e skip connection
71 @register_keras_serializable("MergeRunDual")
72 class MergeRunDual(Layer):
73     def __init__(self, out_channels, **kwargs):
74         super(MergeRunDual, self).__init__(**kwargs)
75         # Primo ramo
76         self.conv1a = Conv2D(out_channels, 3, strides=1, padding='
77             ↪ same')
78         self.relu1a = ReLU()
79         self.conv1b = Conv2D(out_channels, 3, strides=1, padding='
80             ↪ same', dilation_rate=2)
81         self.relu1b = ReLU()
82         # Secondo ramo
83         self.conv2a = Conv2D(out_channels, 3, strides=1, padding='
84             ↪ same', dilation_rate=3)
85         self.relu2a = ReLU()
86         self.conv2b = Conv2D(out_channels, 3, strides=1, padding='
87             ↪ same', dilation_rate=4)
88         self.relu2b = ReLU()
89         # Gogeta
90         self.conv3 = Conv2D(out_channels, 3, strides=1, padding='
91             ↪ same')

```



```

83         self.relu3 = ReLU()
84
85     def call(self, x):
86         branch1 = self.relu1a(self.conv1a(x))
87         branch1 = self.relu1b(self.conv1b(branch1))
88
89         branch2 = self.relu2a(self.conv2a(x))
90         branch2 = self.relu2b(self.conv2b(branch2))
91
92         merged = Concatenate()([branch1, branch2])
93         merged = self.relu3(self.conv3(merged))
94         return Add()([merged, x])
95
96 # CAlayer: Channel Attention Layer
97 @register_keras_serializable("CAlayer")
98 class CAlayer(Layer):
99     def __init__(self, channel, reduction=16, **kwargs):
100         super(CAlayer, self).__init__(**kwargs)
101         self.channel = channel
102         self.reduction = reduction
103         self.conv1 = Conv2D(channel // reduction, 1, strides=1,
104                               ↪ padding='same')
105         self.relu = ReLU()
106         self.conv2 = Conv2D(channel, 1, strides=1, padding='same'
107                               ↪ , activation='sigmoid') #sigmoid: risultato tra 0 e
108                               ↪ 1
109
110     def call(self, x):
111         # Calcolo della media globale per canale
112         y = tf.reduce_mean(x, axis=[1, 2], keepdims=True)
113         y = self.relu(self.conv1(y))
114         y = self.conv2(y)
115         return Multiply()([x, y])
116
117 # Block: combinazione di MergeRunDual, ResidualBlock,
118 ↪ EResidualBlock e CAlayer
119 @register_keras_serializable("Block")
120 class Block(Layer):
121     def __init__(self, out_channels, **kwargs):
122         super(Block, self).__init__(**kwargs)
123         self.merge_run_dual = MergeRunDual(out_channels)
124         self.residual_block = ResidualBlock(out_channels)
125         self.e_residual_block = EResidualBlock(out_channels)
126         self.ca = CAlayer(out_channels)
127
128     def call(self, x):
129         r1 = self.merge_run_dual(x)
130         r2 = self.residual_block(r1)
131         r3 = self.e_residual_block(r2)
132         out = self.ca(r3)

```

```

129         return out
130
131 # Creazione del modello
132 @register_keras_serializable("RIDNET")
133 def RIDNET(n_feats=64, rgb_range=1.0):
134     """
135     n_feats: numero di feature channels usate all'interno del
136         ↪ modello.<br>
137     rgb_range: scala dei valori RGB (ad es. 1.0 se l'input
138         ↪ normalizzato [0,1]).
139     """
140     rgb_mean = (0.4488, 0.4371, 0.4040)
141
142     input_layer = Input(shape=(None, None, 3))
143
144     # Sottosottrai la media (MeanShift con sign=-1)
145     sub_mean = MeanShift(rgb_mean, sign=-1)
146     x = sub_mean(input_layer)
147
148     # Testa: BasicBlock (conv + ReLU)
149     head = Conv2D(n_feats, 3, strides=1, padding='same',
150         ↪ activation='relu')(x)
151
152     # Serie di blocchi
153     b1 = Block(n_feats)(head)
154     b2 = Block(n_feats)(b1)
155     b3 = Block(n_feats)(b2)
156     b4 = Block(n_feats)(b3)
157
158     # Coda: convoluzione finale per ottenere 3 canali
159     tail = Conv2D(3, 3, strides=1, padding='same')(b4)
160
161     # Aggiungi la media (MeanShift con sign=+1)
162     add_mean = MeanShift(rgb_mean, sign=1)
163     res = add_mean(tail)
164
165     # Connessione residua a livello di immagine: somma con l'
166         ↪ input originale
167     output = Add()([res, input_layer])
168
169     model = Model(inputs=input_layer, outputs=output)
170     return model
171
172 model = RIDNET(n_feats=32, rgb_range=1.0)
173 model.compile(optimizer='adam', loss='mse', metrics=['accuracy'])
174     ↪ )
175 model.summary()

```

4 Analisi ed ottimizzazione

4.1 Prestazioni

PSNR è il metodo più comune per misurare la qualità delle immagini.

Il PSNR è definito come il rapporto tra il massimo valore possibile di un segnale e il valore del rumore che disturba la qualità della sua rappresentazione.

Normalmente misurato in una scala logaritmica in decibel(dB).

Data l'immagine originale(g) e l'immagine rumorosa(f), il PSNR è definito come:

$$PSNR = 20 \log_{10} \left(\frac{MAX_f}{\sqrt{MSE}} \right)$$

dove MAX_f è il valore massimo del pixel dell'immagine e si calcola come

$$MSE = \frac{1}{mn} \sum_0^{m-1} \sum_0^{n-1} \|f(i, j) - g(i, j)\|^2$$

mentre MSE (*Mean Square Error*) è l'errore quadratico medio tra l'immagine originale e quella rumorosa.

4.2 Analisi dei modelli

4.2.1 Autoencoder

La rete neurale è un autoencoder convoluzionale simmetrico progettato per la compressione e successiva ricostruzione di immagini RGB.

Nella fase di encoding, l'immagine in input viene inizialmente processata da un primo strato convoluzionale (Conv2D) con 64 filtri, ciascuno di dimensione 3×3 , seguito da un'operazione di sottocampionamento (MaxPooling2D) che riduce la risoluzione spaziale da 112×112 a 56×56 mantenendo invariato il numero di canali. Un secondo strato convoluzionale, anch'esso composto da 64 filtri, approfondisce l'estrazione delle caratteristiche locali, cui segue un ulteriore livello di pooling che porta la risoluzione a 28×28 pixel. A questo punto, l'informazione visiva è compressa in una rappresentazione latente di dimensione (28, 28, 64), che mantiene un buon bilanciamento tra capacità rappresentativa e riduzione della dimensionalità.

La fase di decoding ha l'obiettivo di ricostruire l'immagine originale a partire dalla rappresentazione compressa. In essa, un ulteriore strato convoluzionale affina la mappa delle caratteristiche, seguito da un'operazione di upsampling (UpSampling2D) che ripristina la dimensione spaziale a 56×56 .

Successivamente, un altro strato convoluzionale migliora la qualità della ricostruzione a tale scala. Un secondo upsampling riporta l'immagine alla dimensione originale di 112×112 pixel. Infine, un ultimo strato convoluzionale con 3 filtri produce l'output finale, riconvertendo il volume delle feature maps in un'immagine RGB.

L'architettura è completamente simmetrica rispetto alla fase di encoding e decoding e non prevede strati densi (fully connected), risultando pertanto particolarmente adatta alla gestione di immagini di dimensione variabile. Inoltre, la scelta di mantenere una rappresentazione latente relativamente "larga" ($28 \times 28 \times 64$) rispetto ad autoencoder più tradizionali consente una migliore preservazione dei dettagli visivi, rendendo questo modello particolarmente adatto a compiti di denoising o ricostruzione di immagini degradate.

Table 4.1: Architettura dell'autencoder

Model: functional		
Layer	Output Shape	Params #
Input layer	(None, 112, 112, 3)	0
Conv2D	(None, 112, 112, 64)	1,792
MaxPooling2D	(None, 56, 56, 64)	0
Conv2D 1	(None, 56, 56, 64)	36,928
MaxPooling2D 1	(None, 28, 28, 64)	0
Conv2D 2	(None, 28, 28, 64)	36,928
UpSampling2D	(None, 56, 56, 64)	0
Conv2D 3	(None, 56, 56, 64)	36,928
UpSampling2D 1	(None, 112, 112, 64)	0
Conv2D 4	(None, 112, 112, 3)	1,731
Params		
Total params:	114,307	446.51 KB
Trainable params:	114,307	446.51 KB
Non-trainable params:	0	0.00 B

4.2.2 RIDNet

La rete RIDNet è composta da vari strati che interagiscono in modo complesso, e come si vede dalla tabella ogni passaggio è concatenato in modo sequenziale, dove l'output di un blocco è l'input del successivo ??.

Il flusso di dati inizia con l'Input Layer 5, il quale riceve un'immagine di dimensioni sconosciute (None, None, None, 3). Questo strato non ha parametri addizionali, essendo semplicemente un livello di ingresso.

Subito dopo, il Mean Shift 10 applica una trasformazione ai dati, mantenendo invariata la dimensione dell'immagine. Questo strato ha il compito di centralizzare l'intensità dei pixel.

Il primo blocco convoluzionale della rete è rappresentato dal Conv2D 250, che riduce progressivamente la dimensione spaziale dell'immagine mentre aumenta il numero di filtri da 3 a 32.

Successivamente, il flusso prosegue attraverso una serie di Block che non solo applicano operazioni convoluzionali ma anche attivazioni non lineari, garantendo una maggiore capacità di rappresentazione della rete. Ogni blocco (da Block 20 a Block 23) mantiene una struttura simile e continua ad applicare operazioni convoluzionali mantenendo gli stessi parametri.

L'output dell'ultimo blocco convoluzionale, il Conv2D 299, produce un'immagine di output della stessa dimensione dell'immagine di ingresso, con qualche parametro in meno e nuovamente 3 canali, questa è una prima previsione dell'immagine finale.

Infine, un ulteriore strato di Mean Shift 11 viene applicato per eseguire un altro aggiustamento del valore medio dei pixel nell'immagine di output. Anche in questo caso, non vengono introdotti nuovi parametri.

L'output finale della rete è generato dallo strato Add 413, che esegue una somma tra l'output di Mean Shift 11 e il risultato precedente. Questo step finale non introduce parametri addizionali, ma è cruciale per combinare le informazioni passate attraverso la rete, migliorando la qualità del risultato finale.

In sintesi, la RIDNet si basa su una sequenza di operazioni convoluzionali seguite da trasformazioni di "mean shift" e operazioni di somma, ottimizzando la qualità dell'immagine di output attraverso una struttura profonda e iterativa. La rete è progettata per ridurre progressivamente l'immagine, applicare convoluzioni e migliorare la qualità attraverso un processo di addizione che incorpora informazioni provenienti da differenti livelli della rete.

Table 4.2: Architettura della RIDNet

Architettura della RIDNet. Model: functional_5			
Layer (type)	Output Shape	Param #	Connected to
input_layer_5	(None, None, None, 3)	0	-
mean_shift_10	(None, None, None, 3)	0	input_layer_5[0][0]
conv2d_250 (Conv2D)	(None, None, None, 32)	896	mean_shift_10[0][0]
block_20 (Block)	(None, None, None, 32)	93,666	conv2d_250[0][0]
block_21 (Block)	(None, None, None, 32)	93,666	block_20[0][0]
block_22 (Block)	(None, None, None, 32)	93,666	block_21[0][0]
block_23 (Block)	(None, None, None, 32)	93,666	block_22[0][0]
conv2d_299 (Conv2D)	(None, None, None, 3)	867	block_23[0][0]
mean_shift_11	(None, None, None, 3)	0	conv2d_299[0][0]
add_413 (Add)	(None, None, None, 3)	0	mean_shift_11[0][0]
Params			
Total params:	376,427	1.44 MB	
Trainable params:	376,427	1.44 MB	
Non-trainable params:	0	0.00 B	

4.3 Prima run

Il primo allenamento è stato sottoposto a entrambi i modelli con immagini RGB dal dataset tensorflow "oxford_iiit_pet" con risoluzione 112x112 pixel, un noise_factor di 0.4 e 50 epoche di training ed entrambe con una batch_size di 64.

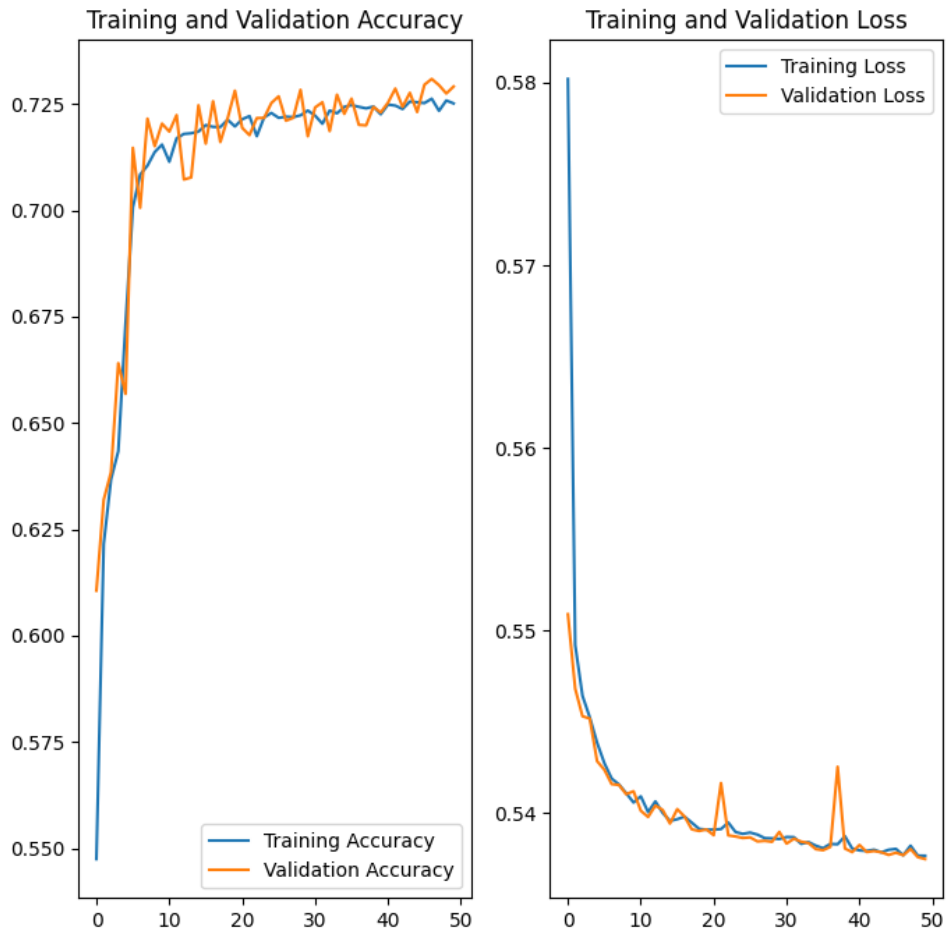
Il dataset non presenta una suddivisione tra train, validation e test ma solo tra train e test, allora è stato suddiviso il training in 80% training e 20% validation.

4.3.1 Autoencoder

Una volta allenato il modello il risultato del training presenta un leggero overfitting sul validation set, mentre sempre nel validation sono presenti un paio di picchi nel loss (come da grafico

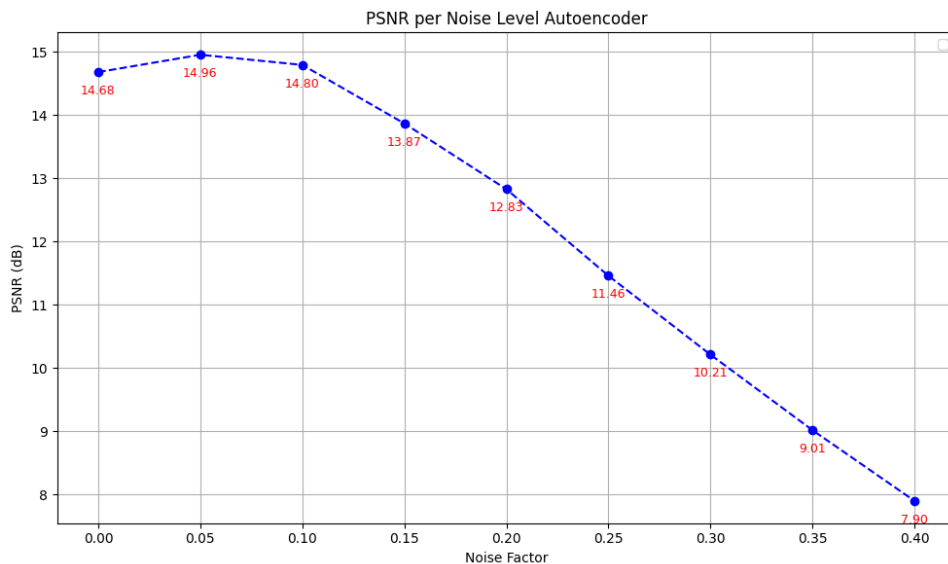
4.1

Figure 4.1: First training of the autoencoder



Poi è stata eseguita una predict e confronto del loss level tra immagini ricreate e reali con diversi livelli di rumore utilizzando la funzione di psnr from skimage.metrics import peak_signal_noise_ratio as psnr

Figure 4.2: Autoencoder psnr level, pet dataset



la media PSNR sul test set è di 5.43 dB. Come si può notare dal grafico??

4.3.2 RIDNet

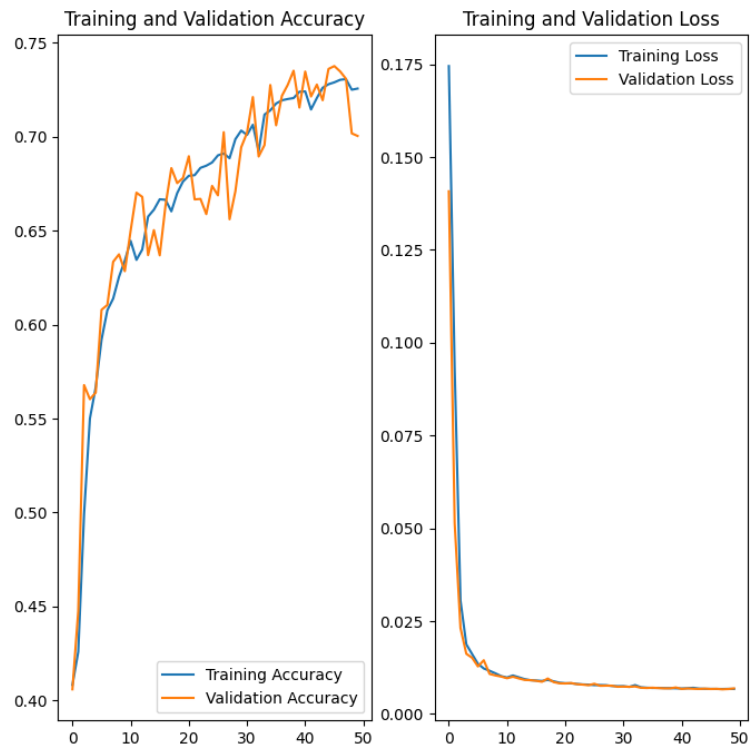


Figure 4.3: First training of the autoencoder

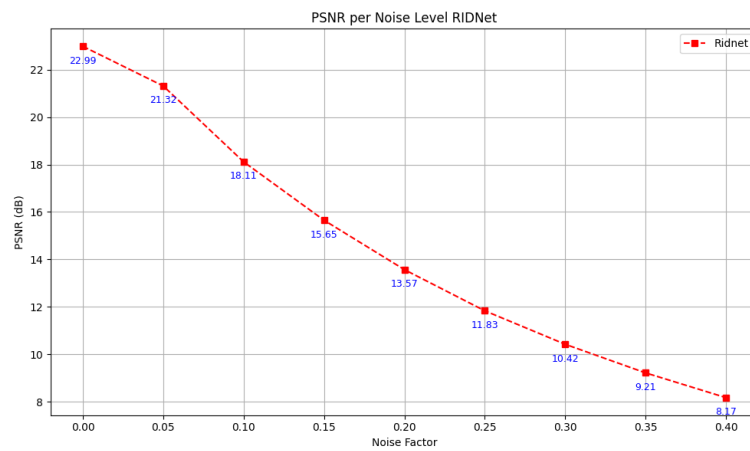


Figure 4.4: First psnr of the autoencoder

la media PSNR sul test set è di 5.37 dB. Come si può notare dal grafico

4.4 Kvasir dataset

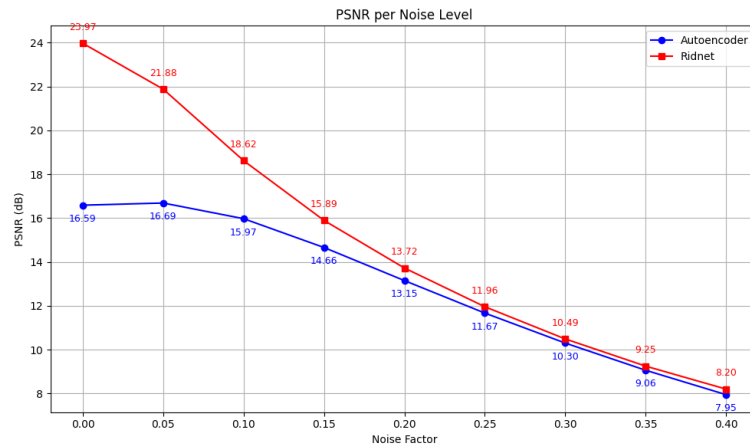


Figure 4.5: PSNR value for kvasir dataset

batch_size più piccola(?) o diverse dimensioni tra split del dataset e train; 50(?) epoche

4.5 Terza run

<QUI SAREBBE DA DOVER CAMBIARE LEARNING RATE E LOSS LEVEL>

4.6 Quarta run

<CAMBIO DELLE ARCHITETTURE DELLE DUE RETI PER MIGLIORARLA A LIVELLO DI PRESTAZIONI>

4.7 Quantizzazione dei modelli

4.7.1 Performance su dataset sconosciuti

Bibliography

- [1] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar. Cats and dogs. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2012.
- [2] Konstantin Pogorelov, Kristin Ranheim Randel, Carsten Griwodz, Sigrun Losada Eskeland, Thomas de Lange, Dag Johansen, Concetto Spampinato, Duc-Tien Dang-Nguyen, Mathias Lux, Peter Thelin Schmidt, Michael Riegler, and Pål Halvorsen. Kvasir: A multi-class image dataset for computer aided gastrointestinal disease detection. In *Proceedings of the 8th ACM on Multimedia Systems Conference*, MMSys'17, pages 164–169, New York, NY, USA, 2017. ACM.